

100 задач на горутины и базовые каналы

1. Первое сообщение

Напиши программу, в которой анонимная горутина отправляет строку "hello" в канал, а main получает и печатает ее.

```
func main()
```

Результат: Одна строка hello. Программа завершается после получения сообщения.

Ограничения: Один небуферизированный канал, одна новая горутина, одна отправка и одно чтение. Канал не закрывай. Отправка является последним действием горутины; main ожидает через чтение канала.

2. Обычная функция в горутине

Реализуй SendNumber и запусти ее через go из main. Передай число и канал аргументами. Получи число в main и напечатай его.

```
func SendNumber(n int, ch chan int)
func main()
```

Результат: При передаче 42 программа печатает 42 один раз.

Ограничения: Одна новая горутина отправляет ровно одно значение в небуферизированный канал и возвращается. Main читает один раз. Канал никто не закрывает; глобальные переменные не используй.

3. Результат анонимной функции

Напиши Double: создай внутри канал и анонимную горутину, которая вычисляет удвоенное значение. Передай n параметром горутины и верни полученный из канала результат.

```
func Double(n int) int
```

Результат: Double(7) возвращает 14; Double(-3) возвращает -6.

Ограничения: Один небуферизированный канал и одна новая горутина. Она отправляет один результат последним действием. Канал не закрывай. Удвоенное значение помещается в int.

4. Приветствие с аргументом

Напиши Greet: передай имя обычной вспомогательной функции, запущенной через go. Она должна составить строку "Привет, " плюс имя и передать ее вызывающей функции.

```
func Greet(name string) string
```

Результат: Для "Иван" верни "Привет, Иван"; для пустого имени верни "Привет, ".

Ограничения: Имя и канал передай аргументами. Одна горутина, один небуферизированный канал, один ответ. После отправки горутина возвращается. Канал никто не закрывает.

5. Сумма в одной горутине

Напиши SumAsync: одна горутина проходит по переданному слайсу, считает сумму и отправляет ее вызывающей функции.

```
func SumAsync(nums []int) int
```

Результат: Для [3, -1, 4] верни 6; для пустого или nil-слайса верни 0.

Ограничения: Даже для пустого слайса отправь один результат. Слайс только читай, сумму храни локально в горутине. Один небуферизированный канал без закрытия; после отправки горутина возвращается. Сумма помещается в int.

6. Длина строки в рунах

Напиши RuneCountAsync: посчитай число рун строки в новой горутине и верни результат через канал.

```
func RuneCountAsync(s string) int
```

Результат: Для "кот" верни 3, для "Go!" верни 3, для пустой строки верни 0.

Ограничения: Вход содержит корректный UTF-8. Используй range по строке или []rune. Одна горутина передает один int последним действием. Канал небуферизированный, закрывать его не нужно.

7. Ожидание после печати

Напиши программу: горутина печатает "готово", затем отправляет true в канал done. Main получает сигнал и только после этого печатает "конец".

```
func main()
```

Результат: Ровно две строки в порядке: готово, затем конец.

Ограничения: Done имеет тип chan bool и не имеет буфера. Одна отправка сигнала, одно чтение. Сигнал отправляй после печати, затем возвращайся из горутин. Канал никто не закрывает; ожидание по времени не используй.

8. Вызов метода через go

Определи Rectangle и метод SendArea. Метод вычисляет площадь и передает ее в канал. Из main запусти метод через go и напечатай полученный результат.

```
type Rectangle struct { Width, Height int }
func (r Rectangle) SendArea(ch chan int)
func main()
```

Результат: Для Rectangle{Width: 3, Height: 5} программа печатает 15.

Ограничения: Стороны неотрицательные, площадь помещается в int. Получатель метода передается по значению. Один небуферизированный канал, одна отправка последним действием метода, одно чтение в main. Канал не закрывай.

9. Структура в ответе

Напиши AnalyzePair: одна горутина вычисляет сумму двух чисел и признак их равенства. Передай оба результата одним значением структуры PairInfo.

```
type PairInfo struct { Sum int; Equal bool }
func AnalyzePair(a, b int) PairInfo
```

Результат: Для 4 и 4 верни PairInfo{Sum: 8, Equal: true}; для 2 и 5 верни {Sum: 7, Equal: false}.

Ограничения: Один канал типа chan PairInfo без буфера. Не передавай поля отдельными сообщениями. После единственной отправки горутина возвращается. Канал никто не закрывает. Сумма помещается в int.

10. Нулевая сумма тоже результат

Напиши SumNonNegative: горутина складывает только неотрицательные элементы слайса. Вызывающая функция получает один результат для любого входа.

```
func SumNonNegative(nums []int) int
```

Результат: Для [-2, 0, 5, -1] верни 5; для [-8, -3] и пустого слайса верни 0.

Ограничения: Не заверши горутину до отправки результата, даже если подходящих чисел нет. Слайс не меняй. Один небуферизированный канал без закрытия, одно чтение. Сумма помещается в int.

11. Три сообщения подряд

Напиши программу: одна горутина последовательно отправляет 10, 20 и 30. Main получает все три числа и собирает их в слайс.

```
func main()
```

Результат: Напечатай слайс [10 20 30] после трех чтений.

Ограничения: Один небуферизированный канал. Отправитель делает ровно три отправки и возвращается. Main делает ровно три чтения. Канал не закрывай; range по каналу здесь не используй.

12. Порядок одного отправителя

Напиши `CopyThroughChannel`: горутина отправляет элементы `nums` по одному в исходном порядке, а вызывающая функция собирает новый слайс.

```
func CopyThroughChannel(nums []int) []int
```

Результат: Для `[4, 4, -1]` верни `[4, 4, -1]`; для пустого входа верни слайс длины 0.

Ограничения: Ровно `len(nums)` отправок и столько же чтений. Один небуферизированный канал без закрытия. Для пустого входа горутину не запускай. Вход не меняй; после последней отправки горутина возвращается.

13. Обратный порядок

Напиши `ReverseThroughChannel`: отправитель проходит по `nums` с конца, получатель добавляет каждое сообщение в новый слайс.

```
func ReverseThroughChannel(nums []int) []int
```

Результат: Для `[1, 2, 3]` верни `[3, 2, 1]`; для пустого входа верни слайс длины 0.

Ограничения: Вход не меняй. Один отправитель передает ровно `len(nums)` значений и возвращается; получатель читает столько же. Небуферизированный канал не закрывай. Для пустого входа сразу верни результат без новой горутины.

14. Сигнал без данных

Напиши программу: горутина печатает числа от 1 до 5 и затем отправляет `struct{}{}` в `done`. `Main` ждет сигнал и печатает "все числа напечатаны".

```
func main()
```

Результат: Числа 1, 2, 3, 4, 5 идут по порядку, каждое на своей строке; последняя строка содержит итоговую фразу.

Ограничения: `Done` имеет тип `chan struct{}{}` без буфера. Числа в канал не отправляй. Единственный сигнал отправляй после всех пяти строк и возвращайся. `Main` читает один раз. Канал никто не закрывает.

15. Два независимых результата

Напиши `TwoResults`: запусти две горутины. Одна отправляет `a`, другая `b`. Получи оба сообщения через общий канал и верни слайс в порядке получения.

```
func TwoResults(a, b int) []int
```

Результат: Для 3 и 8 допустимы `[3, 8]` и `[8, 3]`. Для 3 и 3 верни `[3, 3]`.

Ограничения: Один общий небуферизированный канал, по одной отправке последним действием каждой горутины, ровно два чтения. Канал никто не закрывает. Слайс результата изменяет только получатель.

16. Идентификаторы из цикла

Напиши `CollectIDs`: для каждого числа от 0 до $n-1$ запусти отдельную горутину и передай ей идентификатор аргументом. Собери присланные идентификаторы.

```
func CollectIDs(n int) []int
```

Результат: При $n = 4$ результат содержит 0, 1, 2, 3 по одному разу в любом порядке; при $n = 0$ он пустой.

Ограничения: $0 \leq n \leq 100$. Один общий небуферизированный канал, n отправок и n чтений. Все горутини запусти до сбора результатов; после своей отправки каждая возвращается. Канал не закрывай.

17. Сумма двух половин

Напиши `SumHalves`: раздели `nums` на части `nums[:len(nums)/2]` и `nums[len(nums)/2:]`. Две горутини отдельно считают суммы частей, а получатель складывает два результата.

```
func SumHalves(nums []int) int
```

Результат: Для `[1, 2, 3, 4, 5]` верни 15; для пустого слайса верни 0.

Ограничения: Всегда запускай две горутини, включая случай пустых частей. Каждая передает ровно одну сумму в общий небуферизированный канал и возвращается. Прочитай два значения; канал не закрывай. Вход только читай; суммы помещаются в `int`.

18. Сначала запуск, затем получение

Реализуй `Echo`. Создай небуферизированный канал и передай через него `x` из новой горутини в вызывающую функцию. Устрани зависание схемы, где отправка выполнялась до запуска получателя в той же горутине.

```
func Echo(x int) int
```

Результат: `Echo(12)` возвращает 12 и завершается.

Ограничения: Буфер добавлять нельзя. Отправку выполняет одна новая горутина; чтение выполняет `Echo`. Ровно один обмен, канал никто не закрывает. После отправки новая горутина возвращается.

19. Создай канал перед обменом

Напиши `ReceiveText`: начини с объявления `var ch chan string`, затем правильно инициализируй `ch`, запусти отправителя `text` и верни прочитанную строку.

```
func ReceiveText(text string) string
```

Результат: Для "канал" верни "канал"; для пустой строки верни пустую строку без зависания.

Ограничения: Используй `make` до запуска горутины. Канал небуферизированный; один отправитель, одно сообщение и одно чтение. Отправитель возвращается после отправки. Закрытие канала не требуется.

20. Несколько локальных счетчиков

Напиши `CountSteps`: для каждого значения в `steps` запусти горутину. В ней локальный счетчик увеличивается указанное число раз. Сложи присланные счетчики в вызывающей функции.

```
func CountSteps(steps []int) int
```

Результат: Для `[2, 0, 3]` верни 5; для пустого входа верни 0.

Ограничения: Не более 100 элементов, каждый от 0 до 1000. Каждая горутина отправляет один счетчик в общий небуферизированный канал и возвращается. Получи `len(steps)` сообщений. Канал не закрывай. Общий счетчик из горутин не изменяй.

21. Буфер без новой горутины

Напиши программу: создай канал емкостью 3, отправь в него 4, 5, 6 и только затем получи все значения.

```
func main()
```

Результат: Напечатай `[4 5 6]`. Программа завершается без запуска дополнительных горутин.

Ограничения: Все три отправки должны предшествовать первому чтению. Ровно три чтения; канал не закрывай. Емкость канала строго 3.

22. Заполнение и повторное использование буфера

Напиши программу с каналом емкостью 2. Отправь 1 и 2, прочитай одно значение, отправь 3, затем прочитай оставшиеся два. Сохрани все прочитанное в слайс.

```
func main()
```

Результат: Итоговый слайс `[1 2 3]`. Сразу после первых двух отправок `len(ch)` равен 2, а после всех чтений равен 0.

Ограничения: Дополнительные горутины не запускай. Соблюдай указанный порядок операций. Выполни три отправки и три чтения, канал не закрывай. `Len` используй только для этих последовательных проверок.

23. Точная емкость для набора

Напиши `BufferedCopy`: создай канал емкостью `len(nums)`, сначала отправь в него весь слайс, затем прочитай ровно столько же значений в новый слайс.

```
func BufferedCopy(nums []int) []int
```

Результат: Для `[7, 0, 7]` верни `[7, 0, 7]`; для пустого входа верни слайс длины 0.

Ограничения: Дополнительные горутины не нужны. Вход не меняй. Канал не закрывай; число чтений возьми из `len(nums)`, а не из меняющегося `len(ch)`. Пустой вход не должен приводить к чтению.

24. Маленький буфер для длинной последовательности

Напиши `BufferedSequence`: одна горутина отправляет числа от 1 до `n`, а вызывающая функция одновременно с ее работой получает значения и собирает слайс.

```
func BufferedSequence(n int) []int
```

Результат: При `n = 5` верни `[1, 2, 3, 4, 5]`; при `n = 0` верни пустой результат.

Ограничения: $0 \leq n \leq 100$. Емкость канала строго 2. Ровно `n` отправок и `n` чтений; канал не закрывай. При `n = 0` горутина не запускай. После последней отправки горутина возвращается; буфер не увеличивай до `n`.

25. Пакет из трех полей

Напиши `EncodePoint`: горутина последовательно передает `X`, `Y` и `Z` точки через канал емкостью 1. Получатель собирает массив из трех чисел.

```
type Point3 struct { X, Y, Z int }
func EncodePoint(p Point3) [3]int
```

Результат: Для `Point3{X: 2, Y: -1, Z: 5}` верни `[3]int{2, -1, 5}`.

Ограничения: Не передавай структуру целиком. Одна горутина, три отправки в порядке полей, три чтения. После третьей отправки горутина возвращается. Канал никто не закрывает.

26. Ответ с буфером в одно место

Напиши `StartSquare`: создай канал емкостью 1, запусти вычисление квадрата `n` в горуutine и сразу верни канал. Отдельно напиши `main`, который вызывает функцию и читает ответ.

```
func StartSquare(n int) chan int
func main()
```

Результат: Вызов `StartSquare(6)` дает канал с одним ответом 36; `main` печатает его.

Ограничения: Функция не должна ждать чтения ответа. Горутина отправляет ровно один результат и возвращается, даже если получатель начнет читать позже. Канал никто не закрывает; `main` читает один раз. Квадрат помещается в `int`.

27. Опустоши буфер полностью

Напиши `DrainPrepared`: передай в нее канал, в котором заранее находятся ровно `count` значений. Верни их все в слайсе в порядке чтения.

```
func DrainPrepared(ch chan int, count int) []int
```

Результат: Если заранее отправлены 2, 4, 6 и `count = 3`, верни `[2, 4, 6]`; при `count = 0` верни пустой слайс.

Ограничения: `Ch` создан через `make`, `count >= 0`, `cap(ch) >= count`. Других отправителей и получателей во время вызова нет. Не используй `len(ch)` как границу меняющегося цикла. Закрывать канал не нужно.

28. Буфер не отменяет получение

Реализуй `SumWithBuffer`: горутинка отправляет все элементы `nums` в канал емкостью 1, а вызывающая функция суммирует ровно `len(nums)` полученных значений.

```
func SumWithBuffer(nums []int) int
```

Результат: Для `[8, -3, 2]` верни 7; для пустого входа верни 0.

Ограничения: Не дожидайся отправителя перед началом чтения. При пустом входе сразу верни 0 без горутинки. Канал не закрывай; после последней отправки горутинка возвращается. Вход не меняй, сумма помещается в `int`.

29. Слова разной длины в одном буфере

Напиши `WordBatch`: в последовательной части функции отправь все слова в канал емкостью `len(words)`, затем получи их и верни суммарное число рун.

```
func WordBatch(words []string) int
```

Результат: Для `["Go", "кот", ""]` верни 5; для пустого слайса верни 0.

Ограничения: Строки содержат корректный UTF-8. Дополнительные горутинки не запускай. Одно сообщение на слово, `len(words)` чтений, канал не закрывай. Число сообщений считай по словам, а не по числу рун.

30. Общий буфер для двух отправителей

Напиши `BufferedPair`: две горутинки передают строки `a` и `b` в общий канал емкостью 2. Получи обе строки и верни частоты их появления в мапе.

```
func BufferedPair(a, b string) map[string]int
```

Результат: Для `"a", "b"` верни `{"a": 1, "b": 1}`; для `"a", "a"` верни `{"a": 2}`.

Ограничения: Каждая горутинка отправляет ровно одну строку и возвращается. Получатель читает два сообщения и единолично меняет мапу. Канал никто не закрывает. Порядок прихода строк не фиксирован.

31. Канал только для отправки

Напиши SendGreeting с направленным параметром. Из main создай обычный канал, запусти функцию через go и получи приветствие.

```
func SendGreeting(name string, out chan<- string)
func main()
```

Результат: Для имени "Маша" main печатает "Привет, Маша".

Ограничения: В SendGreeting можно только отправлять. Ровно одна отправка последним действием и одно чтение в main. Канал небуферизированный; никто его не закрывает.

32. Канал только для получения

Напиши ReceiveN с параметром только для чтения. В проверочной программе запусти одну горутину, передающую n целых чисел, и собери их этой функцией.

```
func ReceiveN(in <-chan int, n int) []int
```

Результат: Для последовательности 5, 0, -2 и n = 3 верни [5, 0, -2]; при n = 0 верни пустой слайс.

Ограничения: In создан через make, n >= 0. Единственный отправитель посылает ровно n значений и возвращается. Функция читает ровно n раз, канал не закрывает. Для n = 0 проверке не нужен отправитель.

33. Последнее сообщение и закрытие

Напиши SendAndClose: отправь value в out и закрой канал. В main запусти функцию через go, а затем дважды прочитай канал с comma ok.

```
func SendAndClose(value int, out chan<- int)
func main()
```

Результат: Для value = 9 первое чтение дает 9, true, второе дает 0, false. Напечатай обе пары.

Ограничения: Out небуферизированный и создан через make. У него один отправитель. Только SendAndClose закрывает канал, ровно один раз после отправки; затем возвращается. Main канал не закрывает.

34. Ноль и конец данных

Напиши программу: отправитель передает 0, 5, 0 и закрывает канал. Получатель читает с comma ok и добавляет в слайс только реально отправленные значения.

```
func main()
```

Результат: Напечатай [0 5 0]. Оба переданных нуля должны сохраниться; дополнительный ноль после закрытия не добавляй.

Ограничения: Один небуферизированный канал, один отправитель, три отправки. Закрывает только отправитель после третьего значения и возвращается. Получатель прекращает цикл только при ok == false; range пока не используй.

35. Range до закрытия

Напиши `CollectRange`: получи все числа из `in` через `range` и верни слайс. Для проверки запусти одну горутину, передающую заданный слайс и закрывающую канал.

```
func CollectRange(in <-chan int) []int
```

Результат: Поток 3, 1, 3 дает [3, 1, 3]; канал, закрытый без отправок, дает слайс длины 0.

Ограничения: `In` создан через `make`. Отправитель передает конечное число значений и единолично закрывает канал после последнего, в том числе при пустом входе. Получатель не знает количество заранее и канал не закрывает.

36. Закрытый буфер сохраняет данные

Напиши `ClosedBuffer`: без новых горутин создай канал емкостью `len(nums)`, заполни его, закрой и собери значения через `range` в новый слайс.

```
func ClosedBuffer(nums []int) []int
```

Результат: Для [0, 2, 0] верни [0, 2, 0]; для пустого входа верни слайс длины 0.

Ограничения: Функция сама является единственным отправителем и закрывает канал ровно один раз после заполнения. Чтение начинается после `close`. Вход не меняй. Не используй длину буфера для определения конца чтения.

37. Пустой завершенный источник

Напиши `EmptyInts`: создай канал, закрой его без отправок и верни только для чтения. Напиши `main`, который проходит по нему через `range` и печатает число полученных значений.

```
func EmptyInts() <-chan int
func main()
```

Результат: `Main` печатает 0 и завершается. Возвращенный канал отличается от `nil`.

Ограничения: Новые горутины не нужны. `EmptyInts` единолично закрывает канал до возврата. Никаких значений не отправляй; `main` канал не закрывает.

38. Последовательность до первого отрицательного

Напиши `NonNegativePrefix`: запусти горутину, которая отправляет элементы `nums` до первого отрицательного числа, не включая его. Верни канал для чтения.

```
func NonNegativePrefix(nums []int) <-chan int
```

Результат: Для [2, 0, -1, 7] поток содержит 2, 0; для [-1, 3] поток пустой.

Ограничения: Канал небуферизированный. Отправитель закрывает его на каждом пути завершения, включая пустой вход, и возвращается. Число сообщений равно длине указанного префикса. Получатель обязан дочитать через `range`; вход никто не меняет до конца чтения.

39. Обратный отсчет как источник

Напиши Countdown: функция возвращает канал только для чтения, а одна горутина отправляет числа от n до 1 и закрывает его.

```
func Countdown(n int) <-chan int
```

Результат: При $n = 3$ поток содержит 3, 2, 1; при $n = 0$ сразу заканчивается.

Ограничения: $0 \leq n \leq 100$. Канал без буфера. Отправитель передает ровно n чисел, закрывает канал ровно один раз и возвращается. Получатель читает через range до конца. Не используй паузы между числами.

40. Заккрытие как сигнал завершения

Напиши программу: одна горутина печатает "обработка завершена" и закрывает done. Main ожидает закрытия и затем печатает "выход".

```
func main()
```

Результат: Две строки: сначала обработка завершена, затем выход.

Ограничения: Done имеет тип `chan struct{}` без буфера. Значения в done не отправляй. Закрывает только фоновая горутина после печати, затем возвращается. Main делает одно чтение done и не закрывает его.

41. Сумма потока неизвестной длины

Напиши SumStream: читай все значения in через range и возвращай их сумму только после окончания потока.

```
func SumStream(in <-chan int) int
```

Результат: Поток 5, -2, 0, 4 дает 7; пустой поток дает 0.

Ограничения: In создан через make. В проверке один отправитель передает конечное число значений, затем закрывает канал и возвращается. Получатель количество заранее не знает, канал не закрывает. Сумма помещается в int; буфер может быть любым.

42. Минимум без выдуманного значения

Напиши MinStream: найди минимум среди всех полученных чисел. Второй результат должен сообщать, было ли в потоке хотя бы одно значение.

```
func MinStream(in <-chan int) (int, bool)
```

Результат: Для 8, 3, 9 верни 3, true; для пустого потока верни 0, false.

Ограничения: In не nil. Единственный отправитель закрывает канал после конечной последовательности. Прочитай поток до конца, канал не закрывай. Не используй 0 как начальный минимум непустого потока.

43. Частоты слов

Напиши WordCounts: получай слова и считай количество каждого точного строкового значения в локальной мапе.

```
func WordCounts(in <-chan string) map[string]int
```

Результат: Поток "go", "", "go", "Go" дает {"go": 2, "": 1, "Go": 1}. Пустой поток дает пустую ненулевую мапу.

Ограничения: In не nil. Один отправитель закрывает канал после конечного числа слов и возвращается. дочитай до закрытия. Только WordCounts изменяет мапу; канал не закрывай. Регистр и пустые строки сохраняй.

44. Строка с разделителями

Напиши JoinStream: собери все строки из канала в одну строку с sep между соседними элементами.

```
func JoinStream(in <-chan string, sep string) string
```

Результат: Для потока "a", "", "b" и sep = "," верни "a,,b"; для пустого потока верни "".

Ограничения: In не nil. Единственный отправитель передает конечную последовательность и закрывает канал. дочитай все сообщения, канал не закрывай. Разделитель не добавляй перед первым и после последнего элемента; пустая строка тоже элемент.

45. Уникальные значения в порядке появления

Напиши UniqueStream: верни каждый полученный int только при его первом появлении, сохраняя порядок первых появлений.

```
func UniqueStream(in <-chan int) []int
```

Результат: Поток 3, 1, 3, 2, 1 дает [3, 1, 2]; пустой поток дает слайс длины 0.

Ограничения: In не nil, один отправитель закрывает канал после конечного числа сообщений. дочитай поток целиком. Мапу просмотренных значений и результат меняет только получатель. Канал не закрывай.

46. Самое длинное слово

Напиши LongestWord: найди строку с максимальным числом рун. При равной длине оставь ту, которая пришла раньше. Верни также ее индекс в потоке, начиная с 0.

```
func LongestWord(in <-chan string) (string, int)
```

Результат: Для "кот", "Go", "дом" верни "кот", 0; для пустого потока верни "", -1.

Ограничения: In не nil, строки содержат корректный UTF-8. Один отправитель закрывает канал после конечного числа слов. Получатель читает до конца и канал не закрывает. Поток из одной пустой строки дает "", 0.

47. Сжатие одинаковых соседей

Напиши `Runs`: объедини подряд идущие одинаковые числа в структуры `Run` с самим числом и длиной серии.

```
type Run struct { Value, Count int }  
func Runs(in <-chan int) []Run
```

Результат: Поток 2, 2, 3, 2 дает [{Value: 2, Count: 2}, {Value: 3, Count: 1}, {Value: 2, Count: 1}]. Пустой поток дает пустой слайс.

Ограничения: `In` не `nil`. Один отправитель закрывает канал после конечной последовательности. Обработай последнюю серию при закрытии, не добавляй лишнюю для пустого потока. Канал не закрывай; одинаковые числа из разных серий не объединяй.

48. Проверка порядка с полным чтением

Напиши `IsNonDecreasing`: проверь, что каждое следующее число не меньше предыдущего. После найденного нарушения продолжай получать оставшиеся сообщения.

```
func IsNonDecreasing(in <-chan int) bool
```

Результат: Поток 1, 1, 4 дает `true`; 3, 2, 5 дает `false`. Пустой поток и поток из одного числа дают `true`.

Ограничения: `In` не `nil`. Единственный отправитель передает конечную последовательность и закрывает канал. Верни ответ только после полного чтения, чтобы не оставить отправителя заблокированным. Канал получатель не закрывает.

49. Возьми первые K, дочитай остальные

Напиши `FirstK`: сохрани первые `k` значений потока. Все последующие значения получай и отбрасывай до закрытия канала.

```
func FirstK(in <-chan int, k int) []int
```

Результат: Для потока 5, 6, 7 и `k = 2` верни [5, 6]; при `k = 0` верни пустой слайс после полного чтения.

Ограничения: `K >= 0`, `in` не `nil`. Один отправитель закрывает канал после конечной последовательности. Если значений меньше `k`, верни все. Не выходи из функции сразу после набора `k` значений и не закрывай входной канал.

50. Скользящие суммы по три

Напиши TripleSums: для каждого трех соседних чисел потока вычисли сумму и добавь ее в результат.

```
func TripleSums(in <-chan int) []int
```

Результат: Поток 1, 2, 3, 4, 5 дает [6, 9, 12]; поток короче трех значений дает пустой слайс.

Ограничения: In не nil. Единственный отправитель передает конечное число значений и закрывает канал. Дочитай поток, канал не закрывай. Кроме результата храни не более трех последних чисел. Суммы помещаются в int.

51. Последнее значение ключа

Напиши LastValues: получай пары ключ-значение и собери мапу. Повторная пара с тем же ключом заменяет прежнее значение.

```
type Entry struct { Key string; Value int }  
func LastValues(in <-chan Entry) map[string]int
```

Результат: Поток {"a", 1}, {"b", 2}, {"a", 7} дает {"a": 7, "b": 2}. Пустой поток дает пустую ненулевую мапу.

Ограничения: In не nil. Один отправитель задает порядок, закрывает канал после последней пары и возвращается. Мапу меняет только получатель. Прочитай все сообщения; получатель канал не закрывает.

52. Баланс по последовательности изменений

Напиши BalanceStats: начни с initial и применяй полученные изменения баланса по порядку. Верни конечный баланс и минимальный баланс за всю обработку, включая начальный.

```
func BalanceStats(in <-chan int, initial int) (int, int)
```

Результат: При initial = 10 и потоке -3, -9, 5 верни 3, -2. Для пустого потока верни initial, initial.

Ограничения: In не nil. Все промежуточные значения помещаются в int. Один отправитель закрывает канал после конечной последовательности. Читай до конца; локальный баланс меняет только получатель. Канал не закрывай.

53. Круглые скобки из канала

Напиши `Balanced`: проверь корректность последовательности открывающих и закрывающих круглых скобок. Даже после ошибки дочитай оставшийся поток.

```
func Balanced(in <-chan rune) bool
```

Результат: Для потока символов `"(()())"` верни `true`; для `")("` и `"("` верни `false`; для пустого потока верни `true`.

Ограничения: `In` не `nil`; вход содержит только символы `(` и `)`. Один отправитель закрывает канал после конечного числа рун. Не возвращайся раньше закрытия и не закрывай канал. Храни счетчик глубины и признак ошибки; стек не требуется.

54. Самая длинная серия положительных чисел

Напиши `PositiveStreak`: найди максимальное число подряд идущих значений строго больше 0. Ноль и отрицательное число обрывают серию.

```
func PositiveStreak(in <-chan int) int
```

Результат: Поток `1, 2, 0, 3, 4, 5, -1` дает `3`; пустой поток или поток без положительных значений дает `0`.

Ограничения: `In` не `nil`. Один отправитель передает конечное число сообщений и закрывает канал. Получатель дочитывает весь поток и не закрывает его. Храни только текущую и максимальную длины серий, не сохраняй весь поток.

55. Первое слово без повторов

Напиши `FirstUnique`: после полного чтения потока найди первое по порядку слово, которое встретилось ровно один раз.

```
func FirstUnique(in <-chan string) (string, bool)
```

Результат: Для `"a", "b", "a", "c"` верни `"b", true`; для `"a", "a"` верни `"", false`. Одно пустое слово дает `"", true`.

Ограничения: `In` не `nil`. Один отправитель передает конечное число слов и закрывает канал. Регистр учитывай. Mapy частот и порядок первых появлений хранит только получатель. Канал не закрывай; ответ выдавай после конца потока.

56. Суммы строк матрицы

Напиши `RowSums`: для каждой строки матрицы запусти отдельную горутину. Она передает сумму вместе с индексом строки. Восстанови порядок строк в ответе.

```
func RowSums(rows [][]int) []int
```

Результат: Для `[[1, 2], [], [4, -1]]` верни `[3, 0, 3]`; для пустой матрицы верни пустой слайс.

Ограничения: Не более 100 строк. По одному сообщению `{Index, Sum}` от каждой горутин в общий небуферизированный канал, затем возврат. Получи `len(rows)` сообщений; канал не закрывай. Строки только читай, результат меняет получатель. Суммы помещаются в `int`.

57. Квадраты в исходном порядке

Напиши `OrderedSquares`: запусти по горутине на элемент `nums`. Каждая отправляет его индекс и квадрат. Собери слайс квадратов в порядке исходного входа.

```
func OrderedSquares(nums []int) []int
```

Результат: Для `[4, -2, 3]` верни `[16, 4, 9]`; для пустого входа верни пустой слайс.

Ограничения: Не более 100 элементов; квадраты помещаются в `int`. Индекс и число передавай аргументами. Один общий канал без буфера, `len(nums)` сообщений и чтений, канал не закрывай. Все горутини запусти до сбора; каждая возвращается после отправки. Только получатель записывает итоговый слайс.

58. Поиск первого индекса в двух частях

Напиши `FindFirst`: две горутини ищут `target` в разных половинах `nums`. Каждая возвращает минимальный общий индекс в своей части или `-1`. Выбери минимальный найденный индекс.

```
func FindFirst(nums []int, target int) int
```

Результат: Для `[9, 2, 9, 3]` и `target = 9` верни `0`; для отсутствующего числа и пустого входа верни `-1`.

Ограничения: Раздели вход по `len(nums)/2`. Две горутини, по одному сообщению в общий небуферизированный канал даже для пустой части. Получи оба результата; каждая возвращается после отправки. Канал не закрывай. Вход не меняй и не возвращай первый пришедший ответ.

59. Первый ответ с ожиданием остальных

Напиши `FirstArrival`: запусти по горутине на каждую строку `values`. Верни первую полученную строку, но перед возвратом прими и отбрось все остальные ответы.

```
func FirstArrival(values []string) (string, bool)
```

Результат: Для `["a", "b"]` допустимы `"a", true` или `"b", true`. Для пустого входа верни `""`, `false`.

Ограничения: Не более 100 строк. Каждая горутини отправляет одну строку последним действием в общий небуферизированный канал. Получи ровно `len(values)` сообщений, канал не закрывай. Первый ответ определяется получением, а не порядком запуска; пустой вход не создает горутин.

60. Проверка всех строк без раннего выхода

Напиши AllNonEmpty: каждая горутина проверяет одну строку на непустоту и передает bool. Получатель должен собрать все ответы и вычислить общий результат.

```
func AllNonEmpty(values []string) bool
```

Результат: Для ["a", "b"] верни true; для ["a", "", "b"] верни false; для пустого входа верни true.

Ограничения: Не более 100 строк. Один общий небуферизированный канал, по одному ответу на строку, затем возврат из отправителя. Даже после false прочитай оставшиеся ответы. Канал не закрывай; общий флаг изменяет только получатель.

61. Два разных показателя одной строки

Напиши TextMetrics: одна горутина считает руны, другая считает ASCII-пробелы. Обе передают Metric с разными значениями Name. Собери mapу показателей.

```
type Metric struct { Name string; Value int }
func TextMetrics(s string) map[string]int
```

Результат: Для "a б" верни {"runes": 3, "spaces": 1}; для пустой строки оба значения равны 0.

Ограничения: Строка содержит корректный UTF-8. Считай только пробел U+0020. Один общий небуферизированный канал, ровно два сообщения с Name "runes" и "spaces", затем возврат отправителей. Прочитай оба; канал не закрывай. Mapу собирает получатель.

62. Самый дешевый доступный товар

Напиши Cheapest: отдельная горутина проверяет каждый товар и передает его копию с признаком Stock > 0. Получатель выбирает доступный товар с минимальной ценой, при равенстве цен с меньшим ID.

```
type Product struct { ID, Price, Stock int }
func Cheapest(items []Product) (Product, bool)
```

Результат: Из {ID: 1, Price: 100, Stock: 0}, {ID: 2, Price: 120, Stock: 1}, {ID: 3, Price: 90, Stock: 2} выбери товар 3. Если доступных нет, верни Product{}, false.

Ограничения: До 100 товаров с уникальными ID, Price и Stock неотрицательные. По одному ответу от каждой горутины, затем возврат. Прочитай len(items) сообщений из общего канала без буфера; не закрывай его. Вход не меняй, учитывай все ответы.

63. Локальные частоты и одна итоговая мапа

Напиши CountChunks: каждая горутина считает частоты чисел своего слайса и отправляет отдельную локальную мапу. Получатель объединяет частоты.

```
func CountChunks(chunks [][]int) map[int]int
```

Результат: Для [[1, 2, 1], [2, 3], []] верни {1: 2, 2: 2, 3: 1}. Пустой вход дает пустую ненулевую мапу.

Ограничения: До 100 частей. По одной мапе на часть в общий небуферизированный канал, включая пустые части. После отправки горутина мапу больше не трогает и возвращается. Получи len(chunks) мап; канал не закрывай. Общую мапу изменяет только получатель, вход только читается.

64. Одна запись результата на входной индекс

Напиши NameLengths: для каждого имени горутина отправляет индекс и длину в рунах. Получатель формирует слайс структур, сохраняя исходные имена и порядок, включая повторы.

```
type NameInfo struct { Name string; Runes int }
func NameLengths(names []string) []NameInfo
```

Результат: Для ["Ли", "", "Ли"] верни [{"Ли", 2}, {"", 0}, {"Ли", 2}]. Пустой вход дает пустой слайс.

Ограничения: До 100 корректных UTF-8 строк. По одному сообщению на индекс в общий канал без буфера и по одному чтению на имя. Канал не закрывай; отправители возвращаются после сообщения. Слайс результата заполняет только получатель; мапа с именем в качестве ключа не подходит.

65. Сводка по пустым и непустым частям

Напиши ChunkBounds: каждая горутина находит минимум, максимум и количество чисел своей части. Объедини частичные результаты в общую сводку.

```
type Bounds struct { Min, Max, Count int }
func ChunkBounds(chunks [][]int) Bounds
```

Результат: Для [[], [5, 9], [-2]] верни Bounds{Min: -2, Max: 9, Count: 3}. Если чисел нет, верни Bounds{ }.

Ограничения: До 100 частей. Пустая часть тоже отправляет один Bounds{ } в общий небуферизированный канал. Прочитай len(chunks) сообщений; пустые сводки не учитывай при выборе границ. После отправки горутини возвращаются. Канал не закрывай, вход только читай.

66. Независимая копия структуры

Напиши программу: передай горутине User по значению. Затем main меняет имя у своей переменной. Горутина отправляет имя из полученной копии, а main печатает оба имени.

```
type User struct { ID int; Name string }
func main()
```

Результат: При исходном имени "Иван" и новом имени "Олег" результат горутины равен "Иван", а имя в main равно "Олег".

Ограничения: User передай явным аргументом вызова go; указатель не передавай. Один небуферизированный канал, один ответ последним действием горутины. Main меняет свою копию до чтения ответа. Канал никто не закрывает.

67. Снимок слайса до запуска

Напиши StartSnapshotSum: до запуска горутины сделай независимую копию nums. Горутина суммирует копию и передает один результат через возвращаемый канал.

```
func StartSnapshotSum(nums []int) <-chan int
```

Результат: После вызова для [1, 2, 3] вызывающая сторона может сразу заменить исходные элементы на 100. Ответ должен остаться 6. Для пустого входа ответ 0.

Ограничения: Во время самого вызова вход не меняется. Копируй элементы до go, а не внутри горутины. Емкость результата 1; горутина отправляет один ответ, закрывает свой канал и возвращается. Получатель читает до закрытия. Сумма помещается в int.

68. Снимок карты

Напиши StartMapTotal: синхронно скопируй карту остатков, затем запусти горутину, которая складывает значения копии. Верни канал результата.

```
func StartMapTotal(stock map[string]int) <-chan int
```

Результат: Для {"a": 2, "b": 5} ответ равен 7, даже если исходную карту изменили сразу после возврата функции. Для nil-карты ответ 0.

Ограничения: Во время копирования вызывающая сторона карту не меняет. После возврата фоновая горутина не должна обращаться к исходной карте. Один канал емкостью 1, один ответ, затем закрытие отправителем и возврат. Получатель дочитывает канал. Сумма помещается в int.

69. Структура со слайсом внутри

Напиши `StartCartTotal`: получи `Cart` по значению, отдельно скопируй `Prices` до запуска горютины и вычисли сумму этой копии в фоне.

```
type Cart struct { UserID int; Prices []int }
func StartCartTotal(cart Cart) <-chan int
```

Результат: Для `Prices = [10, 20]` ответ равен 30, даже если после возврата функции исходный `Prices[0]` заменили на 999.

Ограничения: Во время вызова `Cart` и его `Prices` не меняются. Копии заголовка слайса недостаточно: создай отдельное хранилище элементов. Один ответ через канал емкостью 1; отправитель затем закрывает его и возвращается. Пустая корзина дает 0, суммы помещаются в `int`.

70. Независимая копия матрицы

Напиши `StartDoubleMatrix`: до `go` скопируй внешний слайс и каждую строку матрицы. В горютине удвой элементы копии и передай готовую матрицу одним сообщением.

```
func StartDoubleMatrix(rows [][]int) <-chan [][]int
```

Результат: Для `[[1, 2], [], [3]]` получи `[[2, 4], [], [6]]`. Изменения исходных строк после возврата функции не влияют на результат.

Ограничения: Во время копирования вход не меняется. Копии строк не должны разделять память с входом или друг с другом, даже если входные строки разделяли память. Канал емкостью 1; одна отправка, закрытие отправителем и возврат. После отправки он матрицу не трогает. Удвоенные значения помещаются в `int`.

71. Указатель с одним владельцем

Напиши `IncrementOwned` и проверочную программу. `Main` создает `Counter`, запускает функцию через `go` и читает поле `Value` только после сигнала `done`.

```
type Counter struct { Value int }
func IncrementOwned(c *Counter, n int, done chan<- struct{})
func main()
```

Результат: При `Value = 2` и `n = 3` после ожидания поле равно 5. При `n = 0` значение сохраняется, ожидание тоже завершается.

Ограничения: `c` и `done` не `nil`, `n >= 0`, результат помещается в `int`. Во время работы только новая горютина обращается к `Counter`. Она делает `n` увеличений, закрывает `done` последним действием и возвращается. Значений в `done` нет; `main` его не закрывает.

72. Отдельный слайс для каждого сообщения

Напиши Triplets: горутина для каждого i от 0 до $n-1$ готовит $[i, i+1, i+2]$ и отправляет $[]int$. Получатель сохраняет все слайсы и проверяет их независимость.

```
func Triplets(n int) <-chan []int
```

Результат: При $n = 3$ получатель получает $[0, 1, 2]$, $[1, 2, 3]$, $[2, 3, 4]$. Изменение одного полученного слайса не меняет другие.

Ограничения: $0 \leq n \leq 100$. Канал без буфера, ровно n сообщений. Каждый отправленный слайс имеет собственные элементы; отправитель больше их не меняет. После цикла он закрывает канал и возвращается. Получатель дочитывает через range; при $n = 0$ канал закрывается без данных.

73. Снимки изменяемой карты

Напиши WordSnapshots: одна горутина по очереди учитывает слова из words и после каждого слова отправляет независимый снимок текущей карты частот.

```
func WordSnapshots(words []string) <-chan map[string]int
```

Результат: Для $["a", "b", "a"]$ получи $\{"a": 1\}$, затем $\{"a": 1, "b": 1\}$, затем $\{"a": 2, "b": 1\}$. Ранние снимки не изменяются.

Ограничения: Вход не меняется до конца чтения. Канал без буфера, $\text{len}(\text{words})$ сообщений. Для каждой отправки копируй карту по элементам. Рабочую карту меняет только отправитель; отправленные копии он не трогает. После цикла отправитель закрывает канал и возвращается; получатель читает до конца.

74. Метод получает значения до закрытия

Напиши метод Counter.Consume: он получает все числа из in и прибавляет их к Value. В main запусти метод через go, передай изменения и дождись завершения.

```
type Counter struct { Value int }
func (c *Counter) Consume(
    in <-chan int, done chan<- struct{},
)
func main()
```

Результат: При начальном Value = 10 и сообщениях 2, -3, 4 после ожидания Value равен 13. Пустой вход сохраняет Value.

Ограничения: Все указатели и каналы не nil. Main единолично отправляет конечную последовательность и закрывает in. До ожидания done main не читает и не меняет Counter. Метод после range закрывает done и возвращается. Значений в done нет; суммы помещаются в int.

75. Две записи, разделенные сигналом

Напиши `OrderedUpdate`: первая горутина записывает в локальную переменную `x` значение 7. Вторая должна после этого удвоить `x`. Верни результат после завершения второй работы.

```
func OrderedUpdate() int
```

Результат: Функция всегда возвращает 14.

Ограничения: Запусти обе горутины до ожидания результата. Первая закрывает `firstDone` после записи и возвращается. Вторая сначала читает `firstDone`, меняет `x`, закрывает `secondDone` и возвращается. Вызывающая функция читает `x` только после чтения `secondDone`. Оба канала `chan struct{}` без отправки значений; каждый закрывает только назначенная горутина.

76. Один запрос и один ответ

Напиши `RoundTrip`: вызывающая функция отправляет число `n` через `requests`, одна фоновая горутина получает его и передает квадрат через `replies`.

```
func RoundTrip(n int) int
```

Результат: `RoundTrip(5)` возвращает 25; `RoundTrip(0)` возвращает 0.

Ограничения: Два разных небуферизированных канала. Фоновая горутина сначала получает один запрос, затем отправляет один ответ и возвращается. Вызывающая функция сначала отправляет, затем читает. Каналы никто не закрывает; квадрат помещается в `int`.

77. Несколько запросов по очереди

Напиши `DoubleDialogue`: одна фоновая горутина получает числа из `requests` и отвечает удвоенными значениями. Для каждого входного числа отправь запрос и сразу получи ответ.

```
func DoubleDialogue(nums []int) []int
```

Результат: Для `[2, -1, 0]` верни `[4, -2, 0]`; для пустого входа верни пустой слайс.

Ограничения: Два небуферизированных канала. Ровно `len(nums)` запросов и столько же ответов. Вызывающая функция закрывает `requests` после последнего ответа. Фоновая горутина выходит из `range`, закрывает `replies` и возвращается. Вызывающая функция дожидается закрытия `replies` дополнительным чтением с `comma ok`. Удвоенные значения помещаются в `int`.

78. Подтверждение после обработки

Напиши программу: `main` отправляет три строки. Для каждой строки фоновая горутина сначала печатает ее, затем отправляет подтверждение с ее порядковым индексом. `Main` печатает подтверждение перед отправкой следующей строки.

```
func main()
```

Результат: Для строк `a`, `b`, `c` вывод идет парами: `a`, `ask 0`, `b`, `ask 1`, `c`, `ask 2`. Каждая запись печатается на отдельной строке.

Ограничения: Два канала без буфера. Ровно три сообщения и три подтверждения. После третьего подтверждения `main` закрывает канал строк. Получатель закрывает канал подтверждений после выхода из `range` и возвращается. `Main` дожидается его закрытия через `comma ok`; подтверждение отправляй только после печати.

79. Сумма после конца входа

Напиши `FinishSum`: одна фоновая горутина получает все числа, а после закрытия входа передает единственную итоговую сумму. Вызывающая функция отправляет `nums` и возвращает ответ.

```
func FinishSum(nums []int) int
```

Результат: Для `[1, -4, 8]` верни `5`; для пустого входа верни `0`.

Ограничения: Вход и результат без буфера. Вызывающая функция отправляет `len(nums)` чисел и закрывает вход до ожидания результата. Получатель после `range` отправляет одну сумму, закрывает канал результата и возвращается. Вызывающая функция дочитывает результат до закрытия. Вход не меняй; сумма помещается в `int`.

80. Накопительная сумма в ответах

Напиши `RunningTotals`: фоновая горутина владеет локальной суммой. После каждого полученного числа она прибавляет его и отвечает новым значением суммы.

```
func RunningTotals(nums []int) []int
```

Результат: Для `[3, -1, 5]` верни `[3, 2, 7]`; для пустого входа верни пустой слайс.

Ограничения: Одна фоновая горутина и два канала без буфера. На каждый запрос сразу читай один ответ, всего `len(nums)` пар. Затем отправитель закрывает вход. Фоновая горутина закрывает ответы после `range` и возвращается; вызывающая функция дожидается закрытия. Суммы помещаются в `int`.

81. Пакеты с заголовком длины

Напиши `ReadBatches`: поток состоит из пакетов. Сначала идет неотрицательная длина пакета `k`, за ней ровно `k` чисел. Верни все пакеты отдельными слайсами.

```
func ReadBatches(in <-chan int) [][]int
```

Результат: Поток 2, 8, 9, 0, 1, 7 означает пакеты `[[8, 9], [], [7]]`. Пустой поток означает отсутствие пакетов.

Ограничения: `In` не `nil`. Один отправитель передает только корректные пакеты, закрывает канал строго между пакетами и возвращается. Получатель читает заголовок с `comma ok`, затем ровно `k` элементов. Число сообщений равно числу пакетов плюс суммарное число их элементов. Получатель канал не закрывает; слайсы пакетов независимы.

82. Повторное использование буфера с ответом

Напиши `SumBatchesWithAck`: вызывающая функция копирует очередной пакет в один переиспользуемый буфер и передает его слайс фоновой горутине. Та суммирует элементы и отвечает суммой. Только после ответа буфер можно перезаписать.

```
func SumBatchesWithAck(batches [][]int) []int
```

Результат: Для `[[1, 2], [3, 4], [5, 6]]` верни `[3, 7, 11]`. Пустой вход дает пустой слайс.

Ограничения: Все пакеты одинаковой длины, вход не меняется, суммы помещаются в `int`. Один дополнительный буфер, два небуферизированных канала, по одному запросу и ответу на пакет. Фоновая горутина не хранит слайс после ответа. Вызывающая функция закрывает вход; получатель закрывает ответы после `range` и возвращается. Дождись закрытия ответов.

83. Возврат владения указателем

Напиши `ApplyDiscount`: создай копию товара, отправь указатель на нее одной фоновой горутине и получи тот же указатель обратно после уменьшения цены. Верни измененную структуру.

```
type Item struct { ID, Price int }
func ApplyDiscount(item Item, discount int) Item
```

Результат: Для `Item{ID: 1, Price: 100}` и скидки 30 верни `{ID: 1, Price: 70}`. Скидка 150 уменьшает цену до 0.

Ограничения: `Price` и `discount` неотрицательные. Два небуферизированных канала типа `chan *Item`, по одному сообщению в каждом, без закрытия. После передачи указателя вызывающая функция не обращается к копии до ее возврата. Получатель меняет только эту копию, отправляет указатель обратно последним действием и возвращается.

84. Подтвержденный журнал строк

Напиши LogLines: одна горутина получает строки и добавляет их в свой локальный слайс. После закрытия входа она передает готовый слайс вызывающей функции.

```
func LogLines(lines []string) []string
```

Результат: Для ["start", "", "end"] верни отдельный слайс с этими тремя строками в том же порядке. Для пустого входа верни пустой слайс.

Ограничения: Ровно len(lines) сообщений во входном небуферизированном канале. Вызывающая функция закрывает вход после отправок. Получатель отправляет один слайс в канал результата емкостью 1, закрывает его и возвращается; слайс больше не меняет. Вызывающая функция дочитывает результат до закрытия; вход не меняй.

85. Завершение отдельным сообщением

Напиши RunCommands: отправь фоновой горутине по команде на каждое число, затем отдельную команду Stop. На Stop горутина передает итоговую сумму и заканчивает работу.

```
type Command struct { Value int; Stop bool }
func RunCommands(nums []int) int
```

Результат: Для [2, 0, -1] верни 1; для пустого входа отправь только Stop и получи 0. Обычный ноль не завершает обработку.

Ограничения: Канал команд без буфера. Ровно len(nums)+1 команд; Stop отправляется последней. Канал команд закрывает вызывающая функция после отправки Stop. Получатель не ждет этого закрытия: на Stop он отправляет один итог, закрывает канал итогов и возвращается. Канал итогов емкостью 1; дочитай его. Сумма помещается в int.

86. Доступ к данным после сигнала старта

Напиши ReadAfterReady: создай пустую локальную мапу и запусти горутину, которая сначала ждет ready, затем читает ключ "price" и отправляет значение. Заполни мапу после запуска горутины, но до сигнала.

```
func ReadAfterReady(price int) int
```

Результат: ReadAfterReady(120) всегда возвращает 120.

Ограничения: Ready имеет тип chan struct{} без буфера и не содержит значений. Вызывающая функция записывает price, затем закрывает ready и больше мапу не меняет. Горутина обращается к мапе только после чтения ready; отправляет один ответ последним действием и возвращается. Канал ответа без буфера, его никто не закрывает.

87. Строгое чередование реплик

Напиши PingPong: вызывающая функция записывает в свой результат "ping", отправляет сигнал горутине и ждет ответ "pong". Повтори обмен n раз.

```
func PingPong(n int) []string
```

Результат: При $n = 3$ верни ["ping", "pong", "ping", "pong", "ping", "pong"]; при $n = 0$ верни пустой слайс.

Ограничения: $0 \leq n \leq 100$. Два небуферизированных канала: сигнал и ответ. Ровно n обменов; получатель знает n и возвращается после последнего ответа. Каналы не закрывай. Итоговый слайс изменяет только вызывающая функция. При $n = 0$ горутину не запускай.

88. Обмен через один двунаправленный канал

Напиши SameChannelDialogue: вызывающая функция отправляет очередное число через ch , а одна фоновая горутина получает его и отправляет через тот же ch число плюс 1. Собери ответы.

```
func SameChannelDialogue(nums []int) []int
```

Результат: Для [4, 0, -2] верни [5, 1, -1]; для пустого входа верни пустой слайс.

Ограничения: ch строго небуферизированный, других участников нет. Участники знают $\text{len}(\text{nums})$ и чередуют роли: запрос, ответ. Всего $2 \cdot \text{len}(\text{nums})$ отправок и столько же чтений в программе. После последнего ответа фоновая горутина возвращается; канал не закрывай. Пустой вход не запускает горутину, результаты помещаются в int .

89. Остановка источника по подтверждению

Напиши PrefixWithAck: источник отправляет элементы nums по одному и после каждого ждет bool -подтверждение. Получатель сохраняет не более k элементов; после нужного последнего элемента отвечает false , чтобы источник прекратил отправки.

```
func PrefixWithAck(nums []int, k int) []int
```

Результат: Для [1, 2, 3, 4] и $k = 2$ верни [1, 2]; при k больше длины верни весь вход. При $k = 0$ или пустом входе верни пустой слайс.

Ограничения: $k \geq 0$. Для $k = 0$ или пустого входа не запускай горутину. Два канала без буфера; после каждого значения ровно одно подтверждение. False запрещает следующую отправку. При false или исчерпании входа источник закрывает канал данных и возвращается. Получатель дожидается закрытия данных; канал подтверждений никто не закрывает. Вход не меняй.

90. Обработка с итогом и сигналом

Напиши SumAndFinish: фоновая горютина читает числа, затем отправляет итоговую сумму и после этого закрывает done. Вызывающая функция должна получить сумму и дождаться done.

```
func SumAndFinish(nums []int) int
```

Результат: Для [2, 3] верни 5; для пустого входа верни 0. Функция возвращается после итогового сигнала.

Ограничения: Три небуферизированных канала: вход int, результат int и done struct{}. Вызывающая функция отправляет len(nums) чисел и закрывает вход. Получатель после range отправляет ровно одну сумму, закрывает done последним действием и возвращается. Канал результата не закрывай. Сначала читай результат, потом done; суммы помещаются в int.

91. Заккрытие общего канала после двух отправителей

Напиши CollectAndClose: две горютины отправляют элементы своих слайсов a и b в общий канал. После своих отправок каждая сообщает о завершении через finished. Получатель собирает все данные, ждет оба сигнала и закрывает общий канал.

```
func CollectAndClose(a, b []int) []int
```

Результат: Для [1, 2] и [3] верни все три числа по одному разу в порядке получения. При двух пустых слайсах верни пустой результат.

Ограничения: Оба канала небуферизированные. Всего len(a)+len(b) данных и ровно два сигнала struct{}{}, даже для пустых частей. Каждый отправитель после сигнала возвращается и сам ничего не закрывает. Получатель сначала читает все данные, затем оба сигнала, затем закрывает данные и проверяет ok == false. Finished не закрывай; вход не меняй.

92. Сначала завершение отправителей, потом чтение буфера

Напиши BufferedCollectAndClose: две горютины отправляют элементы a и b в общий буфер и сигнализируют через done. Вызывающая функция сначала получает оба сигнала, закрывает данные и только затем собирает их через range.

```
func BufferedCollectAndClose(a, b []int) []int
```

Результат: Для [1, 1] и [2] результат содержит две единицы и одну двойку; порядок между отправителями любой. Пустые входы дают пустой результат.

Ограничения: Емкость данных строго len(a)+len(b), done без буфера. Всего len(a)+len(b) данных и два сигнала. После своего сигнала каждый отправитель возвращается. Только вызывающая функция закрывает данные после двух сигналов; done не закрывай. Вход не меняй. Чтение данных до сигналов не используй.

93. Проверь полноту ответов

Напиши `ValidateReplies`: получи конечный поток индексов и проверь, что каждый индекс от 0 до $n-1$ встретился ровно один раз, без лишних индексов.

```
func ValidateReplies(in <-chan int, n int) bool
```

Результат: При $n = 3$ поток 2, 0, 1 дает true; 0, 1, 1 дает false; 0, 1, 2, 3 тоже дает false. Пустой поток при $n = 0$ дает true.

Ограничения: $N \geq 0$, `in` не nil. Один отправитель закрывает канал после конечного числа сообщений. После любой ошибки дочитай поток до конца. Учти отрицательные индексы, пропуски и повторы. Состояние проверки хранит только получатель; канал не закрывай.

94. Ответ на каждую проверку товара

Напиши `ValidateProducts`: одна горутина получает товары по очереди и отвечает `Validation` на каждый, включая неверные. Товар корректен, если $ID > 0$, `Name` не пустое и $Price \geq 0$.

```
type Product struct { ID int; Name string; Price int }
type Validation struct { ID int; Valid bool }
func ValidateProducts(items []Product) []Validation
```

Результат: Для товаров {1, "чай", 50}, {2, "", 20}, {0, "сок", -1} верни [{1, true}, {2, false}, {0, false}]. Пустой вход дает пустой слайс.

Ограничения: Два небуферизированных канала. На каждый запрос сразу получай один ответ, всего $\text{len}(\text{items})$ пар. Проверка не пропускает ответ при ошибке. Вызывающая функция закрывает вход; проверяющая горутина закрывает ответы после `range` и возвращается. Дождись закрытия ответов. Вход не меняй.

95. Строки для чеков в исходном порядке

Напиши `Receipts`: для каждого заказа запусти горутину, которая формирует строку `"order=<ID>; total=<Total>"` и отправляет ее вместе с индексом заказа. Верни строки в исходном порядке.

```
type Order struct { ID, Total int }
func Receipts(orders []Order) []string
```

Результат: Для [{ID: 7, Total: 100}, {ID: 3, Total: 20}] верни ["order=7; total=100", "order=3; total=20"].

Ограничения: До 100 заказов; $ID > 0$, $Total \geq 0$. Один общий небуферизированный канал результатов, по одному сообщению на заказ. Передавай индекс и `Order` явными аргументами. После отправки каждая горутина возвращается; получатель читает $\text{len}(\text{orders})$ раз и канал не закрывает. Пустой вход дает пустой слайс; разрешен `fmt.Sprintf`.

96. Проверка пользователей с привязкой к входу

Напиши `CheckUsers`: для каждого пользователя отдельная горутина формирует список нарушений. Условия: `ID > 0`, `Name` не пустое, `Age >= 18`. Собери результаты в исходном порядке.

```
type User struct { ID int; Name string; Age int }
func CheckUsers(users []User) [][]string
```

Результат: Для `{ID: 0, Name: "", Age: 16}` список равен `["id", "name", "age"]`; для корректного пользователя список пустой. Количество списков равно количеству пользователей.

Ограничения: До 100 пользователей. Нарушения внутри списка идут в порядке `id`, `name`, `age`. Каждая горутина передает индекс и отдельный слайс одним сообщением, затем больше его не меняет и возвращается. Общий канал без буфера, `len(users)` чтений, без закрытия. Итоговый слайс меняет только получатель; вход не меняй.

97. Баланс с подтверждением операции

Напиши `ApplyDeltas`: одна горутина владеет локальным балансом. Для каждого изменения она либо применяет его, если баланс не станет отрицательным, либо оставляет баланс прежним. После каждой попытки верни подтверждение.

```
type BalanceReply struct { Applied bool; Balance int }
func ApplyDeltas(initial int, deltas []int) []BalanceReply
```

Результат: При `initial = 10` и `deltas = [-7, -5, 4]` верни `{true, 3}, {false, 3}, {true, 7}`. Пустой вход дает пустой слайс.

Ограничения: `Initial >= 0`, вычисления помещаются в `int`. Два небуферизированных канала; один запрос и один ответ на изменение, следующую операцию отправляя после ответа. Вызывающая функция закрывает вход. Получатель закрывает ответы после `range` и возвращается. Дождись закрытия ответов; общий баланс из нескольких горутин не меняй.

98. Снимки корзины после команд

Напиши `CartSnapshots`: одна горутина владеет локальной mapой количеств товаров. Каждая команда меняет количество на `Delta`. Если результат `<= 0`, удали товар. После команды отправь независимый снимок корзины.

```
type CartChange struct { ProductID, Delta int }
func CartSnapshots(changes []CartChange) []map[int]int
```

Результат: Для `{1, 2}, {2, 1}, {1, -2}` получи `{1: 2}`, затем `{1: 2, 2: 1}`, затем `{2: 1}`. Изменение последнего снимка не меняет первые.

Ограничения: `ProductID > 0`, вычисления помещаются в `int`. Два канала без буфера; после каждой отправки команды сразу читай один снимок. Рабочую mapу не передавай, копируй элементы. Вызывающая функция закрывает вход; получатель закрывает снимки после `range` и возвращается. Дождись закрытия. Пустой вход дает пустой слайс.

99. Отчет по событиям заказа

Напиши `SummarizeOrders`: одна горутина получает события и ведет у себя количество заказов и сумму покупок каждого пользователя. После закрытия входа она передает один готовый отчет.

```
type OrderEvent struct { OrderID, UserID, Total int }
type UserSummary struct { Orders, Total int }
func SummarizeOrders(events []OrderEvent) map[int]UserSummary
```

Результат: Для событий {1, 7, 100}, {2, 7, 50}, {3, 8, 20} верни {7: {Orders: 2, Total: 150}, 8: {Orders: 1, Total: 20}}. Пустой вход дает пустую ненулевую карту.

Ограничения: `OrderID` уникальны, ID положительные, `Total` ≥ 0 , суммы помещаются в `int`. Вход без буфера, `len(events)` отправок, затем закрытие вызывающей функцией. Получатель единолично меняет карту; после `range` отправляет один отчет в канал емкостью 1, закрывает его и возвращается. Отчет больше не меняет; вызывающая функция дочитывает результат.

100. Учебный проект: событие о заказе

Добавь в программу маркетплейса одну фоновую горутину для уведомлений. Основная горутина создает три заказа, сохраняет их в своей карте и после каждого сохранения отправляет `OrderEvent`. Обработчик печатает уведомление для каждого события.

```
type OrderEvent struct { OrderID, UserID, Total int }
func NotifyOrders(
    events <-chan OrderEvent, done chan<- struct{},
)
func main()
```

Результат: Для заказов с ID 1, 2, 3 напечатай по строке "order=<ID>; user=<UserID>; total=<Total>" в порядке создания. Последней `main` печатает "обработано 3 события". Все три записи остаются в карте `main`.

Ограничения: `Events` имеет емкость 1. Только `main` меняет карты, отправляет ровно три события и закрывает `events` после них. Передавай структуры со скалярными полями по значению. Обработчик читает через `range`, после всех уведомлений закрывает `done` и возвращается. `Done` без буфера и без значений; `main` ждет его перед итоговой строкой. БД, HTTP и ожидание по времени не используй.